

# AIQT Milestone 11 Build Specification v0.2

**Title:** Issue Lifecycle, Repair Planning, and Controlled Issue Promotion

**Status:** Revision candidate before build handoff

**Depends on:** M10 merged to main, tag `m10-final-review-skills-bootstrap`

**Primary Agent Target:** Claude Code or equivalent coding agent

Build-planning document for implementing AIQT itself. Not a runtime document model for AIQT user projects.

## 1. Source Alignment

M11 extends the M10 baseline. M10 hardened final-review classification and added read-only integration skills planning. M11 adds an issue lifecycle and repair planning layer that lets AIQT manage unresolved issues without turning into a full issue tracker or rewriting checkpoint history.

```
state -> graph -> packet -> checkpoint -> review/export
-> manage/classify -> issue lifecycle -> repair work
```

The governing design remains state-first. `.aiqt/project.json` and `.aiqt/state.json` are the source of truth; `.aiqt/runlog.jsonl` is append-only history; exports are generated views. M11 must reuse M10 classification and must not create a second classifier.

## 2. v0.2 Change Summary

- Removes the non-canonical `M-REPAIR` option. Repair milestones must use normal `M###` IDs, for example `M009`.
- Aligns unknown issue-key exit codes with M9 `review acknowledge`: unknown keys return exit code 3.
- Treats duplicate issue promotion as idempotent success with exit code 0 and the existing `promotedWorkUnitId`.
- Clarifies that M11 commands do not use exit code 10. Missing required arguments/options return exit code 3.
- Defines `validationCommands` sourcing for promoted repair work units.
- Requires complete deterministic JSON examples for all new commands.

## 3. Objective

M11 gives AIQT controlled issue lifecycle and repair controls. It converts M10-classified unresolved issues into manageable workflow objects: listed, accepted, deferred, resolved, moved to post-MVP, or promoted into a bounded repair work unit.

## 4. In Scope

- `aiqt issue list` and `aiqt issue list --json`
- `aiqt issue update <issue-key> --status <status> --reason "..."` and `--json`
- `aiqt repair plan` and `aiqt repair plan --json`
- `aiqt issue promote <issue-key> --title "..."` `--reason "..."` `--validation-command "..."` and `--json`
- Additive issue override state model
- Additive issue promotion/link model
- `manage` and `final-review.md` respect issue overrides and promotion links
- Repair work uses the existing milestone/work-unit schema

## 5. Out of Scope

- `aiqt checkpoint amend`
- `aiqt dependency update` or `graph amend`
- Package publishing
- Automatic skill installation
- Direct AI/API calls

- Hosted backend or dashboard
- Full requirement lifecycle management
- Rewriting historical checkpoints or runlog events

## 6. Repository Implementation Scope

Likely implementation areas:

```
src/cli/commands/issue*.ts
src/cli/commands/repair*.ts
src/core/issues/
src/core/manage/
src/core/export/
src/core/workflow/
src/core/state/
src/core/runlog/
src/core/schema/
tests/unit/
tests/integration/
tests/fixtures/
```

Implementation may differ if the repository already has better module boundaries. Keep changes localized and consistent with existing M1-M10 patterns.

## 7. Runtime and Package Impact

No new runtime package dependency is expected. All commands operate offline against local `.aiqt/` state files. No network calls, external installers, AI-provider SDKs, or package-publishing logic are in scope.

## 8. Schema Contract

### 8.1 Schema version decision

M11 adds optional issue override and promotion structures. This is additive and backward-compatible.

Decision: **do not bump AIQT\_SCHEMA\_VERSION** unless implementation proves unavoidable.

Rules:

- Existing state files without `issues` remain valid.
- Missing `issues`, `issues.overrides`, and `issues.promotions` are treated as empty arrays.
- No historical checkpoint or runlog event is rewritten.

### 8.2 Optional state shape

```
{
  "issues": {
    "overrides": [
      {
        "issueKey": "checkpoint:WU013:issue:user-profile-role-escalation",
        "status": "deferred",
        "reason": "Accepted as post-MVP security hardening before public launch.",
        "updatedAt": "2026-07-14T00:00:00.000Z",
        "sourceCommand": "aiqt issue update"
      }
    ],
    "promotions": [
      {
        "issueKey": "checkpoint:WU025:issue:remaining-form-error-wrappers",
```

```

    "workUnitId": "WU027",
    "milestoneId": "M009",
    "promotedAt": "2026-07-14T00:00:00.000Z",
    "sourceCommand": "aiqt issue promote"
  }
]
}
}

```

### 8.3 Issue statuses

Allowed override statuses:

```

active
accepted
deferred
resolved
post_mvp

```

Meanings:

- **active**: default, no override.
- **accepted**: known limitation accepted for the current lifecycle stage.
- **deferred**: valid issue deferred for later work.
- **resolved**: issue has been addressed outside the original checkpoint history.
- **post\_mvp**: explicitly moved to post-MVP backlog.

Overrides never rewrite original checkpoint issues or review findings.

## 9. Issue Key Contract

M11 reuses the M9/M10 stable key discipline. Issue keys must be deterministic, stable across runs, derived from canonical state fields, and must not use ephemeral display IDs.

Required pattern examples:

```

checkpoint:<workUnitId>:issue:<slug>
review:<findingKey>
checkpoint:WU013:issue:user-profile-role-escalation
checkpoint:WU025:issue:remaining-form-error-wrappers

```

If an issue is derived from a checkpoint issue with no stable slug, derive a deterministic slug from the normalized issue text and source work unit. The same source issue must produce the same issue key across runs.

## 10. Command Contract

### 10.1 aiqt issue list

```

aiqt issue list
aiqt issue list --json

```

Read-only. Lists normalized issues from checkpoint issues, review findings, acknowledged findings, M10 classification buckets, overrides, and promotions. It must not mutate state or append runlog events.

### 10.2 aiqt issue update

```

aiqt issue update checkpoint:WU013:issue:user-profile-role-escalation \
--status deferred \
--reason "Defer until release-hardening repair cycle."

```

Stores or updates an issue override. Reason is mandatory and non-empty after trim. Unknown issue keys return exit code 3.

### 10.3 aiqt repair plan

```
aiqt repair plan
aiqt repair plan --json
```

Read-only. Produces an ordered repair recommendation from active agent-fixable and release-blocking issues. It should recommend `aiqt issue promote ...` commands for promotable issues.

### 10.4 aiqt issue promote

```
aiqt issue promote checkpoint:WU025:issue:remaining-form-error-wrappers \
  --title "Repair remaining form error handling" \
  --reason "Promote agent-fixable accessibility issue into bounded repair work." \
  --validation-command "pnpm validate"
```

Creates a canonical repair work unit using the existing milestone/work-unit schema. Duplicate promotion is idempotent success and returns the existing `promotedWorkUnitId`.

## 11. Repair Work Graph Rules

### 11.1 Repair milestone ID

Repair milestones must use the existing milestone ID generator and canonical `M###` format. Do **not** use `M-REPAIR`.

If no repair milestone exists:

```
create next canonical milestone ID, e.g. M009
title: Review Repair Work
```

If a repair milestone already exists, reuse it.

### 11.2 Promoted work-unit requirements

Promoted work units must have non-empty:

- title
- objective
- scope
- outOfScope
- acceptanceCriteria
- validationCommands

Promoted work units use the next canonical `WU###` ID.

### 11.3 validationCommands sourcing

Sourcing order:

1. Use one or more supplied `--validation-command` values.
2. Else use `project.quality.preferredValidationCommands` if non-empty.
3. Else block promotion with exit code 2 and tell the user to provide `--validation-command`.

M11 must **not** guess generic validation commands when none are available.

### 11.4 Status and recommendation

A promoted work unit may be **ready** only if all readiness gates pass, including non-empty `validationCommands` and satisfied blocking dependencies. Recalculate milestone status, project status, and `nextRecommendedCommand` after promotion.

## 12. Classification and Override Precedence

M11 must not create a second classifier. The effective issue view is built in this order:

1. canonical checkpoint/review data
2. M10 classification
3. M11 overrides
4. M11 promotion links

Rules:

- **resolved** issues should no longer appear as active release blockers.
- **deferred** and **post\_mvp** issues remain visible but move to the appropriate section.
- **accepted** issues remain visible as accepted limitations.
- Promoted issues should show their promoted work unit ID.

## 13. nextRecommendedCommand Contract

`state.nextRecommendedCommand` remains exactly one command string. Multi-step guidance belongs in reason text.

Issue promotion can make a repair work unit ready, so normal next-action precedence applies after promotion.

## 14. Exit Code Contract

M11 commands do not use exit code 10. Missing required arguments/options return exit code 3.

| Command                         | Exit 0                                         | Exit 2                                                                   | Exit 3                                                                   | Exit 10  |
|---------------------------------|------------------------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|----------|
| <code>aiqt issue list</code>    | listed                                         | n/a                                                                      | invalid project/state                                                    | not used |
| <code>aiqt issue update</code>  | override stored or idempotent no-op            | issue exists but transition not allowed                                  | unknown key, invalid status, empty reason, invalid state                 | not used |
| <code>aiqt repair plan</code>   | plan generated                                 | n/a                                                                      | invalid project/state                                                    | not used |
| <code>aiqt issue promote</code> | created or already promoted idempotent success | issue exists but not promotable, or <b>validationCommands</b> unresolved | unknown key, invalid ti-<br>tle/reason/validation command, invalid state | not used |

## 15. Runlog Contract

Append runlog events only for mutation commands:

- `issue.updated`
- `issue.promoted`

Events must conform to the canonical `RunlogEventSchema`: `id`, `type`, `timestamp`, `actor`, `summary`, `relatedIds`, `data`.

### 15.1 `issue.updated` example

```
{
  "id": "EVT-...",
  "type": "issue.updated",
  "timestamp": "2026-07-14T00:00:00.000Z",
  "actor": "cli",
  "summary": "Updated issue checkpoint:WU013:issue:user-profile-role-escalation to deferred",
  "relatedIds": ["WU013"],
```

```

    "data": {
      "issueKey": "checkpoint:WU013:issue:user-profile-role-escalation",
      "status": "deferred",
      "reason": "Defer until release-hardening repair cycle.",
      "sourceCommand": "aiqt issue update"
    }
  }
}

```

## 15.2 issue.promoted example

```

{
  "id": "EVT-...",
  "type": "issue.promoted",
  "timestamp": "2026-07-14T00:00:00.000Z",
  "actor": "cli",
  "summary": "Promoted WU025 issue to work unit WU027",
  "relatedIds": ["WU025", "WU027", "M009"],
  "data": {
    "issueKey": "checkpoint:WU025:issue:remaining-form-error-wrappers",
    "workUnitId": "WU027",
    "milestoneId": "M009",
    "title": "Repair remaining form error handling",
    "sourceCommand": "aiqt issue promote"
  }
}

```

Do not append runlog events for issue list or repair plan.

## 16. JSON Output Contracts

### 16.1 issue list --json

```

{
  "issues": [
    {
      "issueKey": "checkpoint:WU013:issue:user-profile-role-escalation",
      "source": "checkpoint",
      "workUnitId": "WU013",
      "status": "active",
      "classification": ["release_blocker", "agent_fixable"],
      "message": "user_profile RLS allows self-role escalation",
      "promotedWorkUnitId": null
    }
  ],
  "counts": {
    "active": 1,
    "accepted": 0,
    "deferred": 0,
    "resolved": 0,
    "post_mvp": 0,
    "promoted": 0
  }
}

```

### 16.2 issue update --json

```

{
  "issueKey": "checkpoint:WU013:issue:user-profile-role-escalation",

```

```

"status": "deferred",
"reason": "Defer until release-hardening repair cycle.",
"changed": true,
"nextRecommendedCommand": "aiqt manage"
}

```

### 16.3 repair plan --json

```

{
  "recommendedRepairs": [
    {
      "issueKey": "checkpoint:WU025:issue:remaining-form-error-wrappers",
      "title": "Repair remaining form error handling",
      "promotable": true,
      "recommendedCommand": "aiqt issue promote <issue-key>"
    }
  ]
}

```

### 16.4 issue promote --json

```

{
  "issueKey": "checkpoint:WU025:issue:remaining-form-error-wrappers",
  "promoted": true,
  "alreadyPromoted": false,
  "workUnitId": "WU027",
  "milestoneId": "M009",
  "nextRecommendedCommand": "aiqt next"
}

```

## 17. Test Matrix

### 17.1 Unit tests

- Issue key generation is deterministic.
- Missing `issues` fields are treated as empty arrays.
- Override precedence over M10 classification is deterministic.
- Promotion link lookup is deterministic.
- Repair milestone ID generation uses `M###` only.
- `validationCommands` sourcing blocks when no command is available.

### 17.2 Integration tests

- `issue update` writes state and runlog event.
- `issue promote` creates repair milestone/work unit and runlog event.
- `manage` and `final-review` respect overrides.
- `repair plan` is read-only and deterministic.

### 17.3 CLI contract tests

- Unknown issue key returns exit 3.
- Empty reason returns exit 3.
- Duplicate promotion returns exit 0 and existing `workUnitId`.
- Non-promotable issue returns exit 2.
- Promotion without `validationCommands` returns exit 2.

## 17.4 Regression tests

- Natural Medicine terminal-state fixture with release blockers and agent-fixable issues.
- Resolved issue no longer counts as active release blocker.
- `post_mvp` issue moves to backlog section.
- Promoted issue links to new repair work unit.

## 18. Definition of Done

M11 is complete when:

- `aiqt issue list` works in human and JSON modes.
- `aiqt issue update` stores auditable overrides and respects exit-code rules.
- `aiqt repair plan` produces deterministic read-only recommendations.
- `aiqt issue promote` creates canonical repair work units using `M###` and `WU###` IDs.
- Duplicate promotion is idempotent success.
- Promoted work units always have non-empty `validationCommands` or promotion blocks.
- `manage` and `final-review` respect issue overrides and promotions.
- No `M-REPAIR` ID exists anywhere in state.
- No M11 command uses exit code 10.
- Runlog events use canonical `RunlogEventSchema`.
- All existing tests pass and M11 tests cover the dogfood-derived failure modes.

## 19. Agent Execution Packet

Implement M11 exactly as scoped. Do not implement checkpoint amendment, dependency update, graph repair, skill installation, package publishing, external API calls, or hosted services. Preserve state-first design and append-only history.

Return an implementation report with:

- branch and commit hash;
- files changed;
- commands added;
- schema changes;
- runlog event types added;
- issue key strategy;
- promotion/milestone ID behavior;
- validation command sourcing behavior;
- exit-code behavior;
- tests added and final test count;
- validation results;
- deviations from spec;
- deferred/unresolved items.